Day 82 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 6 reference

Watch: Backpropagation calculus (3Blue1Brown) | Building micrograd (Andrej Karpathy)

`day82_transformer_backward.py`, segment by segment, in the order `main()` uses things. Everything up to section 4 is Day 81's code, copied verbatim (see Day 81's syntax guide). Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: G = the gradients, by name · dy = gradient at a decoder block's input · denc = gradient at the encoder's output · dx = gradient inside the encoder · E = the shared embedding table

Segment 1 — The backward pass

TrainableTransformer.forward: remembering the inputs

CODE

```
class TrainableTransformer(Transformer):
    def forward(self, src, smask, tgt_in, tmask):
        self.src, self.tgt_in, self.smask, self.tmask = src, tgt_in, smask, tmask
        self.logp = super().forward(src, smask, tgt_in, tmask)
        return self.logp
```

- `class TrainableTransformer(Transformer)` — inherits Day 81's model; only forward is extended and gradients added.
- `super().forward(...)` — Day 81's forward pass, unchanged.
- `self.src, self.tgt_in, ... = ...` — four attributes in one line: the backward pass needs the token ids for the embedding table.

gradients: the chain, in reverse

MATH

$$denc = \Sigma_j denc_j; \quad dE = scatter\text{-}add(dy, target\ ids) + scatter\text{-}add(dx, source\ ids)$$

CODE

```
def gradients(self, y, broken=False):
    m, T = self.tgt_in.shape
    mask = self.tmask.ravel()
    dlog = np.exp(self.logp); dlog[np.arange(y.size), y.ravel()] -= 1
    dlog *= mask[:, None] / mask.sum()
    G = dict(Wout=self.dec_out.reshape(-1, D).T @ dlog, bout=dlog.sum(axis=0))
    dy = (dlog @ self.P["Wout"].T).reshape(m, T, D)
    denc = np.zeros_like(self.enc_out)
    for i in reversed(range(len(self.dec))):
        dy, de, g = self.dec[i].backward(dy)
        denc = de if broken else denc + de
        G.update({f"d{i}.{k}": v for k, v in g.items()})
    G["E"] = np.zeros_like(self.P["E"])
    np.add.at(G["E"], self.tgt_in, dy)
    dx = denc
    for i in reversed(range(len(self.enc))):
        dx, g = self.enc[i].backward(dx)
        G.update({f"e{i}.{k}": v for k, v in g.items()})
    np.add.at(G["E"], self.src, dx)
    return G
```

- `dlog[np.arange(y.size), y.ravel()] -= 1` — softmax minus one-hot, in place: subtract 1 at each true token.
- `for i in reversed(range(len(self.dec)))` — the last decoder block first.
- `denc = de if broken else denc + de` — the correct line adds every block's share; broken=True overwrites (Section 3).
- `np.add.at(G["E"], self.tgt_in, dy)` — the target side of the table; later the source side is added to the same array.

train_step

MATH

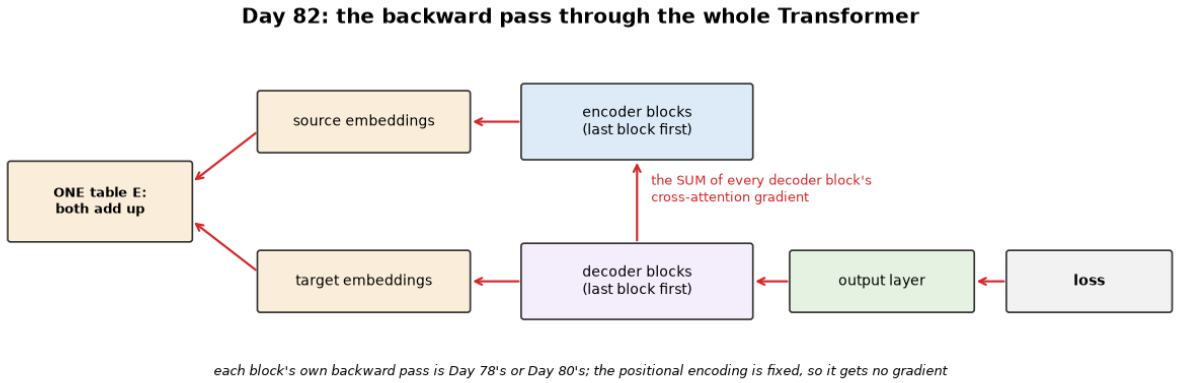
$$w \leftarrow w - lr \cdot min(1, clip / |G|) \cdot G$$

CODE

```
def train_step(self, src, smask, tgt_in, tmask, y, lr, clip=1.0):
    self.forward(src, smask, tgt_in, tmask)
    G, P = self.gradients(y), self.params()
    norm = np.sqrt(sum(np.sum(v ** 2) for v in G.values()))
    for k, v in G.items():
        P[k] -= lr * min(1.0, clip / norm) * v
    return norm
```

- `min(1.0, clip / norm)` — Day 60's clipping: shrink the step only if the gradient is too long.
- `P[k] -= ...` — params() holds the very arrays inside the blocks, so -= updates them.

SYSTEM DIAGRAM



The backward pass, right to left.

Segment 2 — The checks

gradient_check, by parameter group

CODE

```
def gradient_check(net, batch, broken=False, eps=1e-5):
    src, sm, tgt_in, tm, y = batch
    net.forward(src, sm, tgt_in, tm); G = net.gradients(y, broken)
    groups = {}
    for k, A in net.params().items():
        worst, ok = 0.0, True
        for pos in np.ndindex(A.shape):
            old = A[pos]
            A[pos] = old + eps; up = net.loss(src, sm, tgt_in, tm, y)
            A[pos] = old - eps; dn = net.loss(src, sm, tgt_in, tm, y)
            A[pos] = old
            num = (up - dn) / (2 * eps)
            worst = max(worst, abs(num - G[k][pos])); ok &= abs(num - G[k][pos]) <= 1e-10 + 1e-6 *
                abs(num)
        g = k if k in ("E", "Wout", "bout") else k.split(".")[0]
        w, o = groups.get(g, (0.0, True)); groups[g] = (max(w, worst), o and ok)
    return groups
```

- `for pos in np.ndindex(A.shape)` — every index tuple of the array, in order.
- `k.split(".")[0]` — 'e0.Wq' becomes 'e0': the group is the block.
- `groups.get(g, (0.0, True))` — a default for a group seen for the first time.
- `o and ok` — a group passes only if every array in it passed.

tiny_batch: padding on both sides

CODE

```
def tiny_batch():
    rng = np.random.RandomState(3)
    src = rng.randint(1, 9, size=(3, 5)); tgt_in = np.concatenate([np.full((3, 1), GO),
        rng.randint(1, 9, size=(3, 4))], axis=1)
    y = rng.randint(0, V, size=(3, 5))
    sm = np.ones((3, 5), bool); sm[1, 3:] = False; sm[2, 4:] = False
    tm = np.ones((3, 5), bool); tm[1, 4:] = False
    return src, sm, tgt_in, tm, y
```

- `sm[1, 3:] = False` — the second example's source is 3 digits long: positions 3 and 4 are padding.
- `tm[1, 4:] = False` — one padded target position: it must not count in the loss.

demo_gradient_check: tiny sizes, two blocks per side

CODE

```
global D, F, HEADS
saved = (D, F, HEADS); D, F, HEADS = 8, 12, 2
net = TrainableTransformer(n_enc=2, n_dec=2, seed=1, out_scale=0.1)
for b in net.enc + net.dec:
    for k in b.P:
        if k[0] in "gb" and k[1:].isdigit():
            b.P[k] = b.P[k] + 0.1 * np.random.RandomState(7).randn(D)
n = sum(v.size for v in net.params().values())
groups = gradient_check(net, tiny_batch())
```

- `global D, F, HEADS` — rebinds the module-level sizes; Section 3 puts them back.
- `k[0] in "gb" and k[1:].isdigit()` — names like 'g1' or 'b3': the LayerNorm weights.
- `n = sum(v.size for v in ...)` — how many entries the check covers.

Segment 3 — Training sanity

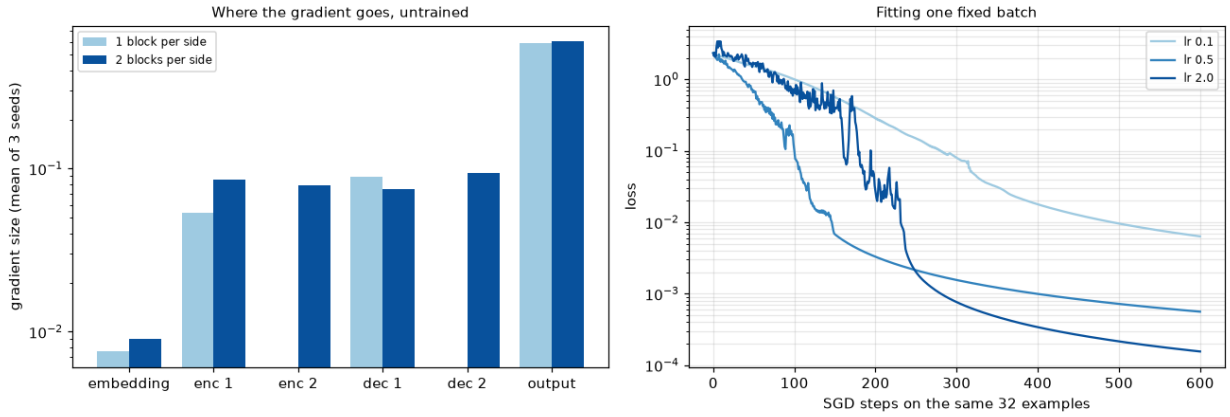
demo_overfit: one fixed batch

CODE

```
for lr in (0.1, 0.5, 2.0):
    net = TrainableTransformer(seed=0, out_scale=0.1)
    losses = [net.loss(src, sm, tgt_in, tm, y)]
    for step in range(1, 601):
        net.train_step(src, sm, tgt_in, tm, y, lr)
        losses.append(net.loss(src, sm, tgt_in, tm, y))
    acc = (net.forward(src, sm, tgt_in, tm).argmax(axis=1).reshape(y.shape) == y).mean()
    curves[lr] = losses
```

- `losses = [net.loss(...)]` — a list started with the loss before any step.
- `argmax(axis=1).reshape(y.shape) == y` — the predicted token at every position, compared with the target.

REAL GRAPH



Real graph from the script: gradient size by part, and fitting one batch.

Quick Syntax Glossary — new or reinforced constructs

- `class B(A): def f(self): ... super().f()` — extend a method and still run the original.
- `a = x if cond else a + x` — overwrite or accumulate, chosen by a flag.

- `np.add.at(A, ids, rows)` — scatter-add: repeated ids add up.
- `np.ndindex(shape)` — every index tuple of an array.
- `s.split('.')[0]` — the text before the first dot.
- `dict.get(key, default)` — a value, or a default if the key is missing.
- `global name` — rebind a module-level name inside a function.

End of Day 82 syntax reference. For the experiments and what they showed, see the Day 82 complete guide.